

Final Project: Part 2 - Secure Information Using Symmetric Encryption



Estimated time needed: 30 minutes

In **Part 2** of the final project, you will demonstrate your ability to protect **SecureBank's** information by implementing appropriate symmetric encryption techniques. As part of this project, you will also review a peer's project submission using the rubric provided.

Part 2 consists of one task:

- **Task 4:** Securing information using symmetric encryption

Step-by-step instructions

Refresh your knowledge

Before starting on Part 2 of this project, you can refresh your knowledge by referring to the following lesson:

- **Module 5, Lesson 1:** Introduction to Cryptography

Project scenario

Imagine you have recently been hired as an ethical hacker by SecureBank, a mid-sized financial institution known for its rapidly growing digital services. In the wake of recent incidents in the banking sector, the management has become increasingly anxious about potential vulnerabilities in SecureBank's online banking platform. Preliminary investigations have highlighted weak security measures, including inadequate cryptographic protections for sensitive financial data. To address these risks, the bank has tasked you with strengthening its security posture through advanced cryptographic techniques.

Objectives

In this assignment, you will implement symmetric encryption to secure sensitive information at SecureBank by:

- Encrypting financial data using symmetric encryption techniques (e.g., AES)
- Decrypting encrypted data to ensure it can be accessed securely
- Managing encryption keys properly to protect against unauthorized access
- Demonstrating the implementation of encryption and decryption in a secure environment

Optional: After completing each task, save both the executed commands and their corresponding outputs in a text document, or take screenshots—depending on the option you have chosen. This information will be required to answer questions during the final project submission.

About Skills Network Cloud IDE

Skills Network Cloud IDE (based on Theia and Docker) provides an environment for hands-on labs for course and project-related labs. Theia is an open-source IDE (Integrated Development Environment) that can be run on a desktop or on the cloud. To complete this lab, you will be using the Cloud IDE based on Theia, running in a Docker container.

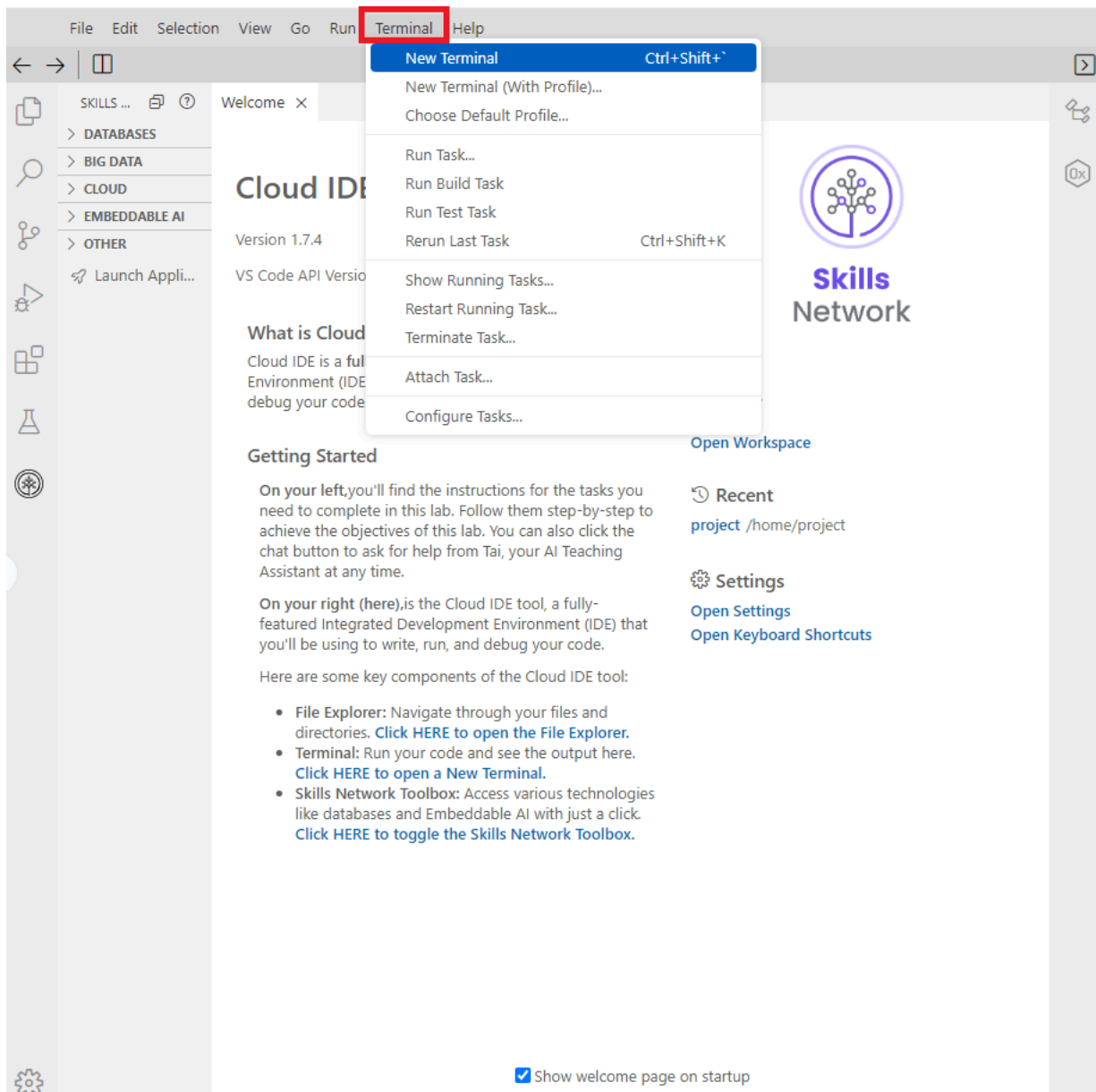
Important notice about this lab environment

Please be aware that sessions for this lab environment are not persistent. A new environment is created for you every time you connect to this lab. Any data you may have saved in an earlier session will get lost. To avoid losing your data, please plan to complete these labs in a single session.

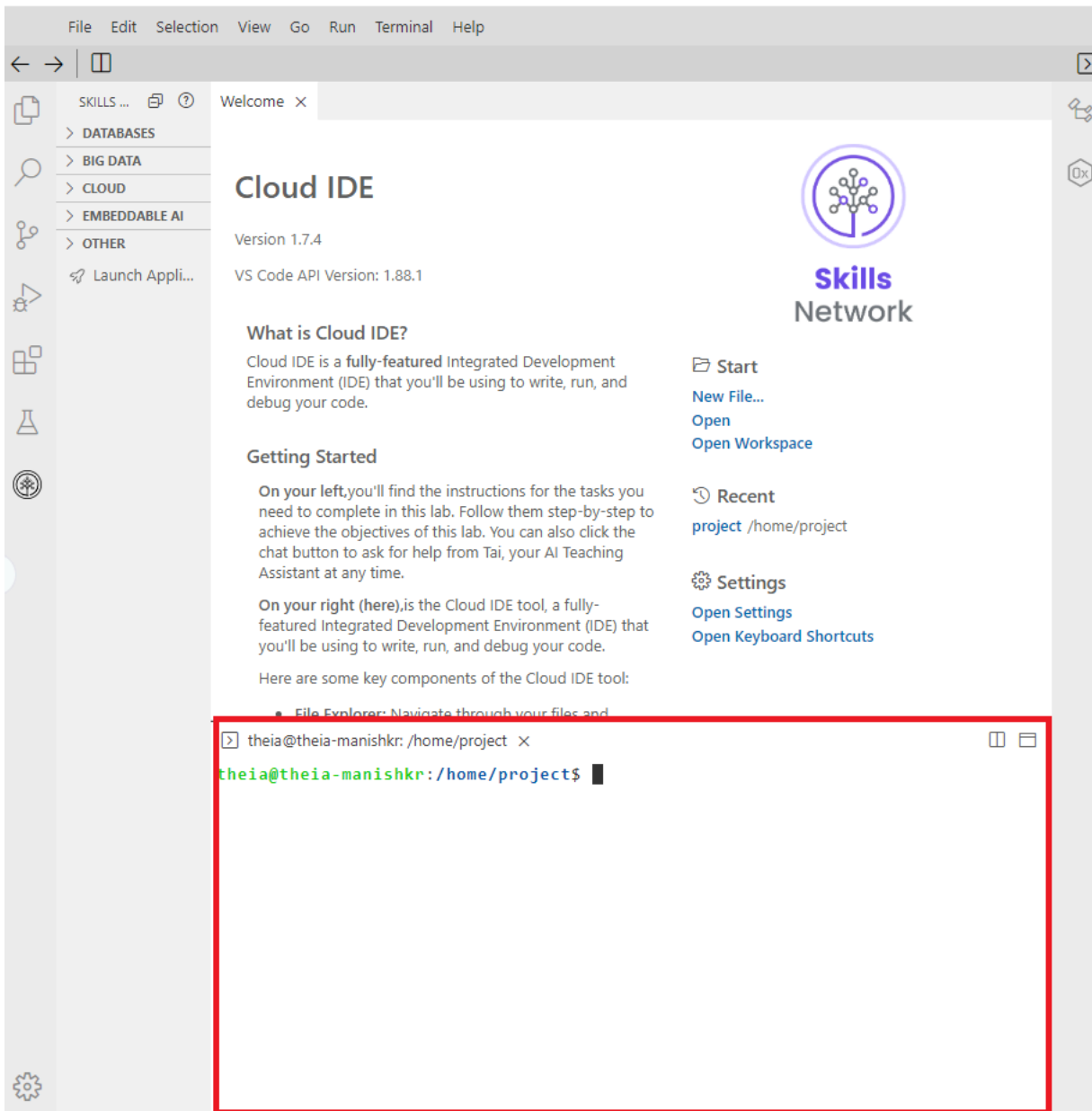
Set up the lab environment

Instructions:

- Open a new terminal by selecting **Terminal**, then choosing **New Terminal** from the drop-down menu.
- Alternatively, you can select **Click HERE to open a New Terminal** under the **Getting Started** section.



- Observe that the new terminal opens. You can complete the six steps in task 4 in this terminal. Remember to capture screenshots on the go.



Secure information using symmetric encryption

Task scenario: During the penetration testing exercise, you discovered a confidential file named userdetails on a web server. The file contains important credentials stored in plain text format, which poses a significant security risk, as unauthorized individuals could easily access them. Your task is to secure this information by implementing proper encryption techniques and demonstrating how to manage sensitive data securely.

This task consists of six steps. Complete each step and upload the required screenshot in the designated area.

Sample credentials:

You are provided with the following credentials that need to be stored in the file, userdetails:

Username

admin

Password:

P@ssw0rd123

Email:

admin@example.com

Phone:

123-456-7890

Here are the steps to secure this sensitive information:

Step 1: Create the file, userdetails

- **Hint:**
 - Create the userdetails file using any Linux-based tools/commands.
 - Display the content of the file using the cat command.

Step 2: Generate the Advanced Encryption Standard (AES) random key

- **Hint:** Create a random encryption key for AES encryption and store it in the passkey file.

Step 3: Encrypt the userdetails file using passkey

- **Hint:** Use the generated key to encrypt the contents of the userdetails file and create an encrypted file named userdetails_encrypt.

Step 4: Display the contents of the encrypted file using the cat command

- **Hint:** Display the contents of the userdetails_encrypt file, even though it is in an unreadable binary format.

Step 5: Decrypt the userdetails_encrypt file using passkey

- **Hint:** Use the same key to decrypt the previously encrypted file userdetails_encrypt and store the output in a new file named userdetails_decrypt.

Step 6: Display the contents of the decrypted file

- **Hint:** Use a suitable command to display the contents of the userdetails_decrypt file and verify that the credentials match those in the original userdetail's file.

Screenshot checklist

You should have the following screenshots with .jpg or .png extension:

- **Step 1:** Screenshot displaying the contents of the userdetails file.
- **Step 2:** Screenshot displaying the successful execution of the key generation command and a list of files confirming the creation of the passkey file.
- **Step 3:** Screenshot displaying the successful execution of the encryption command and the file listing confirming the creation of the userdetails_encrypt file.
- **Step 4:** Screenshot displaying the encrypted content of the userdetails_encrypt file.
- **Step 5:** Screenshot displaying the successful execution of the decryption command and confirming the creation of the userdetails_decrypt file.
- **Step 6:** Screenshot displaying the decrypted contents of the userdetails_decrypt file.

This action concludes the assignment.

Author



Skills Network